

大规模深度学习模型训练中的 计算图 (DAG) 调度优化算法 结合前瞻树搜索与局部搜索的混合策略

数据结构

NJU

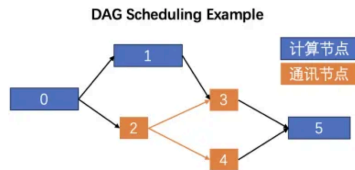
2025 年 12 月

目录

- ① 项目背景与问题定义
- ② 核心算法：Look Ahead Tree Search
- ③ 后处理优化：Local Search
- ④ 实验结果
- ⑤ 总结

背景：大模型训练的效率瓶颈

- **现状：**随着 Transformer 架构模型（如 GPT-4）规模扩大，显存限制使得分布式训练成为必选项。
- **痛点：**GPU 利用率（MFU）通常低于 43%，主要瓶颈在于通信开销。
- **目标：**通过**计算通信重叠（Overlap）**技术，最大化计算资源利用率。
- **挑战：**
 - **节点量级大：**如 Llama 7B 有上万个节点。
 - **约束复杂：**存在复杂的依赖关系与同类节点互斥。
 - **难度极高：**典型的 NP-Hard 问题。



图：神经网络计算图示例

问题建模

我们将深度学习模型的训练过程建模为有向无环图 $G = (V, E)$:

节点与边

- **计算节点 (Compute)**: 执行矩阵运算, 占用计算单元。
- **通讯节点 (Communication)**: 执行 AllGather/ReduceScatter, 占用带宽。
- **互斥约束**: 同类型节点 (主要是通讯节点) 共享资源, 不能同时执行。

优化目标

最小化整个 DAG 的执行跨度 (**Makespan**)。

现有方法的局限

- **MILP (混合整数规划)**: 精确但耗时呈指数级增长, 无法处理万级节点。
- **HEFT (启发式)**: 贪心策略, 容易陷入局部最优。

算法一：基于前瞻的树搜索 (Look Ahead Tree Search)

为了克服贪心算法的短视，我们引入了具备“前瞻”能力的树搜索策略。

核心思想： 在每一步决策时，不只看当前最优，而是模拟未来几步的执行情况，选择综合收益最高的节点。

优先级设计 (Hybrid Priority):

$$\text{Priority}(n) = \text{Rank}(n) - \text{EFT}(n)$$

- **Rank (HEFT):** 距离终点的关键路径长度（全局视角）。
- **EFT:** 最早完成时间（局部视角）。
- 结合两者以适应大模型计算图特殊的拓扑结构。

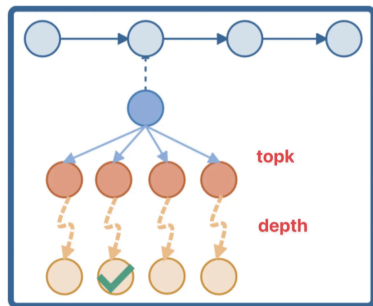


图: Top-K 模拟前瞻示意图

树搜索算法流程详解

① 候选筛选 (Top-K Candidates):

- 获取当前所有入度为 0 的就绪节点。
- 根据 Priority 排序, 选取前 K 个作为候选节点。

② 并行模拟 (Simulate Greedy Schedule):

- 对每个候选节点, 假设选中它。
- **Copy State**: 复制当前调度状态 (时间表、资源)。
- **Simulate**: 基于贪心策略继续模拟调度 d 层 (Depth)。
- 此过程使用线程池并行加速。

③ 评估与决策:

- 计算效率比率 (Efficiency Ratio), 选择 Ratio 最小者:

$$\text{Ratio} = \frac{\text{Simulated Makespan} - \text{Initial Makespan}}{\text{Total Cost of Scheduled Nodes}}$$

- 该指标表示 “单位成本带来的时间增长”, 值越小越好。

算法复杂度分析

该算法主要通过 Top-K 和模拟深度 d 来平衡效果与速度。

复杂度

$$O(n^2 \cdot k)$$

- n 为节点数量。
- k 为 Top-K 参数（常数）。
- 相比于 MILP 的指数级复杂度，该算法在工程上是可行的，且优于简单的 $O(n^2)$ 贪心算法。

构造解的问题：气泡 (Bubbles)

问题观察：即便使用了树搜索，由于贪心策略的本质，生成的调度方案中仍可能存在微小的 **Gap (气泡)**，导致资源浪费。Trace 分析显示，通讯节点往往构成了调度的瓶颈。



图：调度结果中的气泡示例（通讯节点间的空隙）

算法二：局部搜索优化 (Local Search)

在树搜索得到初始解 S 后，通过对**通讯节点**进行微小扰动来进一步压缩 Makespan。

搜索策略

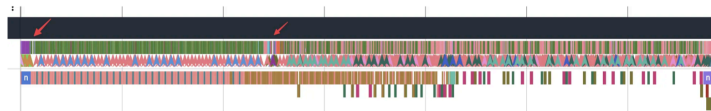
- **对象**：关键路径上的 Communication 类型节点。
- **邻域定义**：对于节点 n ，尝试将其移动到当前位置前后 **WindowSize (w)** 范围内的所有可能位置。

执行流程

- ① **迭代**：执行 I 次大循环。
- ② **并行探索**：对每个候选移动位置，并行计算新的调度 Makespan。
- ③ **更新**：如果发现 $NewMakespan < BestMakespan$ ，则更新解。

局部搜索效果展示

迭代前的结果



迭代后的结果

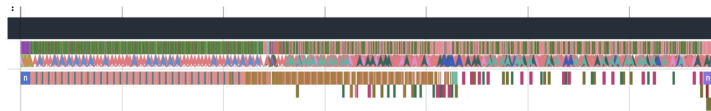


图: 迭代优化效果: 红色箭头处的气泡被消除, 排列更紧凑

数值结果对比 (Llama 7B)

我们将该算法（代号 0320）与基线（XLA, OR-Tools）及 Megatron 进行了对比。

配置	Baseline (XLA)	OR-Tools	TreeSearch Only	Tree+Local (0320)	Megatron
TP=1 DP=8	0.61s	0.52s	0.48s	0.47s	0.51s
Memory	40.95GB	48.31GB	49.26GB	50.12GB	79GB
TP=2 DP=4	0.69s	0.58s	0.58s	0.47s	0.52s
Memory	40.97GB	45.18GB	45.18GB	55.75GB	60GB

表: 不同并行策略下的迭代时间 (s/iter) 与显存占用对比

- **性能提升**: 相比 XLA 提升显著 ($0.61s \rightarrow 0.47s$), 且优于 Megatron。
- **消融实验**: Local Search 进一步降低了 Makespan ($342859 \rightarrow 341662$)。

- **核心贡献：**提出了一种结合 **前瞻树搜索**与 **局部搜索**的两阶段 DAG 调度算法。
- **关键创新：**
 - Rank-EFT 混合优先级，兼顾全局关键路径与局部资源约束。
 - 引入模拟机制 (Simulation) 跳出贪心算法的局部最优。
 - 利用局部搜索 (Local Search) 针对性消除通信气泡。
- **实际收益：**在 Llama 7B 等大模型训练场景中，实现了比 XLA 更高效的计算通信重叠，显著缩短了训练迭代时间。